```
import os
os.chdir('/content/drive/My Drive/')
```

```
import os
print(f"Current Directory: {os.getcwd()} | Files: {os.listdir()}")
```

```
Current Directory: /content/drive/My Drive | Files: ['VFS CODE.gdoc', 'Colab Notebooks', 'THE SMART VFS ROUTER: D
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
```

```
import os

# 1. Force the directory to the Colab 'Home' where uploads live
os.chdir('/content/')

# 2. Synchronize the file system (Forces the sidebar to talk to the kernel)
os.sync()

# 3. Print the 'Truth' - This list MUST show your iodata.docx
print("--- ENVIRONMENT SYNCED ---")
print(f"Current Path: {os.getcwd()}")
print(f"Detected Files: {os.listdir()}")
print("--------------------------")
```

```
--- ENVIRONMENT SYNCED ---
Current Path: /content
Detected Files: ['.config', 'read.txt', 'band.zip', 'mp.png', 'gate.png', 'tsp.jpeg', 'pnp.jpeg', 'iodata.pdf', '
--------------------------
```

```
import os

# 1. Check the temporary area
print("Checking /content/:", os.listdir('/content/'))

# 2. Check your actual Google Drive
if os.path.exists('/content/drive/MyDrive'):
    print("Checking MyDrive:", os.listdir('/content/drive/MyDrive')[:10]) # First 10 files
else:
    print("Drive folder exists but MyDrive subfolder is not found.")

# 3. Use the 'find' command (The absolute truth)
print("\n--- SYSTEM SEARCH RESULTS ---")
!find /content -name "iodata.pdf"
```

```
Checking /content/: ['.config', 'read.txt', 'band.zip', 'mp.png', 'gate.png', 'tsp.jpeg', 'pnp.jpeg', 'iodata.pdf
Checking MyDrive: ['VFS CODE.gdoc', 'Colab Notebooks', 'THE SMART VFS ROUTER: DEFINITIVE MASTER BLUEPRINT.gdoc',

--- SYSTEM SEARCH RESULTS ---
/content/iodata.pdf
```

```
import os
# Force reset the path
os.chdir('/content')
!rm -rf /content/NVMe_WAL /content/HDD_Archive
print(f"Directory Reset. Current Path: {os.getcwd()}")
print(f"Available Files: {os.listdir()}")
```

```
Directory Reset. Current Path: /content
Available Files: ['.config', 'read.txt', 'band.zip', 'mp.png', 'gate.png', 'tsp.jpeg', 'pnp.jpeg', 'iodata.pdf',
```

```
# Force-remove the folders and files at the system level
!rm -rf /content/NVMe_WAL
!rm -rf /content/HDD_Archive
!rm -f /content/iodata.docx
!rm -f /content/*.tmp

# Reset the working directory
import os
os.chdir('/content')

print("--- CLEAN SLATE ---")
print(f"Current Files: {os.listdir('/content')}")
```

```
--- CLEAN SLATE ---
Current Files: ['.config', 'read.txt', 'band.zip', 'mp.png', 'gate.png', 'tsp.jpeg', 'pnp.jpeg', 'iodata.pdf', 'c
```

```
import os
if os.path.exists('/content/iodata.docx'):
    print("✅ VERIFIED: iodata.docx is physically present and accessible.")
    os.chdir('/content')
else:
    print("❌ STILL MISSING: Please re-upload to the sidebar.")
```

```
❌ STILL MISSING: Please re-upload to the sidebar.
```

```
import os

target = "iodata.pdf"

if os.path.exists(f"/content/{target}"):
    print(f"✅ SYSTEM ALERT: {target} is ALIVE.")
    print(f"Copy this exact string for your engine: {os.path.abspath(target)}")
else:
    print("❌ SYSTEM ALERT: File is still invisible to Python.")
    print("Action: Re-upload the file and do NOT refresh the page.")
```

```
✅ SYSTEM ALERT: iodata.pdf is ALIVE.
Copy this exact string for your engine: /content/iodata.pdf
```

[VFS-ARCHIVE-01] CSV-INFRASTRUCTURE-VETTING

```
import hashlib
import os
import zlib
import struct
import math
import time

class VFSSovereignArchitect:
    def __init__(self):
        self.magic_sig = b"VFS!"

    def _get_slab_size(self, size):
        GB, TB, PB = 1024**3, 1024**4, 1024**5
        if size < GB: return 65536
        if size < TB: return 1048576
        if size < PB: return 67108864
        return 1073741824

    def trigger_save(self, path, ram_data, tier_limit, tier_name, storage_mode):
        orig_size = len(ram_data)
        if orig_size == 0:
            print("\n[!] ERROR: Input is 0 bytes. Aborting."); return

        # START PRECISION TIMER
        start_time = time.perf_counter()

        slab_size = self._get_slab_size(orig_size)
        level = 1 if storage_mode == "HOT" else 9
        chunks = []
        cumulative_entropy = 0
        master_hasher = hashlib.sha256()

        for i in range(0, orig_size, slab_size):
            chunk = ram_data[i : i + slab_size]
            master_hasher.update(chunk)
            c_hash = hashlib.sha256(chunk).digest()
            compressed_payload = zlib.compress(chunk, level=level)
            cumulative_entropy += (len(compressed_payload) / len(chunk)) * 100
            chunks.append({"payload": compressed_payload, "hash": c_hash})

        avg_ent = cumulative_entropy / len(chunks)
        map_tax = (len(chunks) * 44) + 48
        final_v_size = 4 + sum(len(x['payload']) for x in chunks) + map_tax
        net_saving = orig_size - final_v_size

        is_efficient = (avg_ent <= tier_limit and net_saving > 0)
        route = "VFS (Vault)" if is_efficient else "DIRECT (Raw)"

        with open(path, "wb") as f:
            if is_efficient:
                f.write(self.magic_sig)
```

```
                    f.write(self.magic_sig)
                    map_metadata = []
                    for x in chunks:
                        pos = f.tell()
                        f.write(x['payload'])
                        map_metadata.append((pos, len(x['payload']), x['hash']))
                    map_start_ptr = f.tell()
                    for off, plen, chash in map_metadata:
                        f.write(struct.pack(">QI32s", off, plen, chash))
                    f.write(struct.pack(">QQ32s", map_start_ptr, orig_size, master_hasher.digest()))
                else:
                    f.write(ram_data)
                f.flush()
                os.fsync(f.fileno())

            # STOP PRECISION TIMER
            end_time = time.perf_counter()
            latency = end_time - start_time
            # Calculate Throughput (MB per second)
            throughput = (orig_size / (1024*1024)) / latency if latency > 0 else 0

            print("\n" + "-"*60)
            print(f" FILE NAME      : {os.path.basename(path)}")
            print(f" ORIGINAL SIZE  : {orig_size:,} bytes")
            print(f" AVG ENTROPY    : {avg_ent:.2f}%")
            print(f" TIER/MODE      : {tier_name} / {storage_mode}")
            print(f" ROUTE TAKEN    : {route}")
            print(f" WRITTEN SIZE   : {os.path.getsize(path):,} bytes")
            if is_efficient:
                gain_pct = (net_saving / orig_size) * 100
                print(f" NET ECONOMY    : {net_saving:,} bytes ({gain_pct:.2f}%)")
            print(f" LATENCY        : {latency:.4f} seconds")
            print(f" THROUGHPUT     : {throughput:.2f} MB/s")
            print("-"*60)

        def trigger_open(self, path):
            if not os.path.exists(path): return

            start_time = time.perf_counter()
            with open(path, "rb") as f:
                v_data = f.read()

            if not v_data.startswith(self.magic_sig):
                print(f"[INFO] {path} is RAW. Standard restoration bypassed.")
                return

            try:
                map_ptr, orig_size, m_hash_exp = struct.unpack(">QQ32s", v_data[-48:])
                num_chunks = (len(v_data) - 48 - map_ptr) // 44
                master_verify = hashlib.sha256()

                with open(path, "wb") as out:
                    for i in range(num_chunks):
                        idx = map_ptr + (i * 44)
                        off, plen, hexp = struct.unpack(">QI32s", v_data[idx:idx+44])
                        chunk = zlib.decompress(v_data[off:off+plen])
                        if hashlib.sha256(chunk).digest() != hexp:
                            raise ValueError(f"Integrity failure at chunk {i}")
                        master_verify.update(chunk)
                        out.write(chunk)
                    out.flush()
                    os.fsync(out.fileno())

                end_time = time.perf_counter()
                latency = end_time - start_time
                throughput = (orig_size / (1024*1024)) / latency if latency > 0 else 0

                if master_verify.digest() != m_hash_exp:
                    print("[CRITICAL] Master Integrity Violation!")
                else:
                    print(f"\n[SUCCESS] {path} Restored (Bit-Perfect).")
                    print(f" RESTORE LATENCY: {latency:.4f} seconds")
                    print(f" RESTORE SPEED  : {throughput:.2f} MB/s")
            except Exception as e:
                print(f"\n[CRITICAL FAIL] Restoration Aborted: {e}")

    def main():
        vfs = VFSSovereignArchitect()
        print("\n>>> VFS SOVEREIGN SYSTEM ONLINE <<<")
        while True:
            print("\n [1] SAVE   [2] OPEN   [3] EXIT")
            cmd = input("Command: ").strip()
            if cmd == "3": break
```

```
            fn = input("Target Filename: ").strip()
            if cmd == "1":
                if not os.path.exists(fn):
                    print(f"File '{fn}' not found."); continue
                with open(fn, "rb") as f: plasma = f.read()
                t_choice = input("Select Tier ([1] Enterprise-80% | [2] Executive-70%): ").strip()
                m_choice = input("Select Mode ([1] HOT | [2] COLD): ").strip()
                limit = 80.0 if t_choice == "1" else 70.0
                name = "ENTERPRISE" if t_choice == "1" else "EXECUTIVE"
                mode = "HOT" if m_choice == "1" else "COLD"
                vfs.trigger_save(fn, plasma, limit, name, mode)
            elif cmd == "2":
                vfs.trigger_open(fn)

    if __name__ == "__main__":
        main()
```

```
Target Filename: band.zip
Select Tier ([1] Enterprise-80% | [2] Executive-70%): 1
Select Mode ([1] HOT | [2] COLD): 1

------------------------------------------------------------
 FILE NAME      : band.zip
 ORIGINAL SIZE  : 96,037 bytes
 AVG ENTROPY    : 72.42%
 TIER/MODE      : ENTERPRISE / HOT
 ROUTE TAKEN    : VFS (Vault)
 WRITTEN SIZE   : 67,719 bytes
 NET ECONOMY    : 28,318 bytes (29.49%)
 LATENCY        : 0.0099 seconds
 THROUGHPUT     : 9.26 MB/s
------------------------------------------------------------

 [1] SAVE   [2] OPEN   [3] EXIT
Command: 2
Target Filename: band.zip

[SUCCESS] band.zip Restored (Bit-Perfect).
 RESTORE LATENCY: 0.0061 seconds
 RESTORE SPEED  : 14.96 MB/s

 [1] SAVE   [2] OPEN   [3] EXIT
Command: 1
Target Filename: child.docx
Select Tier ([1] Enterprise-80% | [2] Executive-70%): 1
Select Mode ([1] HOT | [2] COLD): 1

------------------------------------------------------------
 FILE NAME      : child.docx
 ORIGINAL SIZE  : 107,154 bytes
 AVG ENTROPY    : 98.81%
 TIER/MODE      : ENTERPRISE / HOT
 ROUTE TAKEN    : DIRECT (Raw)
 WRITTEN SIZE   : 107,154 bytes
 LATENCY        : 0.0091 seconds
 THROUGHPUT     : 11.19 MB/s
------------------------------------------------------------

 [1] SAVE   [2] OPEN   [3] EXIT
Command: 2
Target Filename: child.docx
[INFO] child.docx is RAW. Standard restoration bypassed.

 [1] SAVE   [2] OPEN   [3] EXIT
Command: 1
Target Filename: gate.png
Select Tier ([1] Enterprise-80% | [2] Executive-70%): 1
Select Mode ([1] HOT | [2] COLD): 1

------------------------------------------------------------
 FILE NAME      : gate.png
 ORIGINAL SIZE  : 127,161 bytes
 AVG ENTROPY    : 82.22%
 TIER/MODE      : ENTERPRISE / HOT
 ROUTE TAKEN    : DIRECT (Raw)
 WRITTEN SIZE   : 127,161 bytes
```